

Samenvatting – Mobile IoT Apps (Angular/React + ASP.NET + state + build pipeline)

Deze samenvatting is een **onderwerpen-overzicht** (zonder PDF-verwijzingen) over dezelfde leerstof: Angular (met Material + SCSS), React, NgRx/Redux, ASP.NET Web API (EF + MongoDB) en de moderne frontend build pipeline.

Studietip: focus op (1) begrippen, (2) de standaard flow/architectuur, (3) “wanneer gebruik ik wat?”, (4) kleine codepatronen die je kan herkennen/uitschrijven.

1) Angular (met Material Design)

1.1 Setup & tools

- **Node.js + npm/yarn:** basis om packages te installeren en scripts te runnen.
- **Angular CLI:**
 - Install: `npm install --global @angular/cli`
 - Nieuw project (klassiek met routing): `ng new <naam> --routing`
 - Material toevoegen: `ng add @angular/material`
- **Projectstructuur (belangrijkste config):**
 - `package.json`: scripts + dependencies
 - `angular.json`: build/serve options
 - `tsconfig.json`: TypeScript config
 - `src/index.html`, `src/main.ts`
 - `src/app/*`: app-code (componenten, services, routing, ...)

Wat je hieruit moet kunnen uitleggen:

- Wat de CLI voor je automatiseert (scaffolding, build, serve, test).
- Waar je “globale” settings staan (`angular.json`, `tsconfig.json`).

1.2 Modules vs standalone components

- **Klassiek (modules):** Angular gebruikt `@NgModule` om dependencies te bundelen.
 - `imports`: andere modules die je nodig hebt
 - `declarations`: componenten/directives/pipes die bij de module horen
 - `bootstrap`: root component
- **Standalone components:** nieuwere aanpak om module-complexiteit te verminderen.
 - Dependencies importeer je **in de component**.
 - Klassiek project afdwingen: `ng new --standalone=false`

1.3 Componenten (de kern van Angular)

- Een component is een **custom HTML element** (selector) met:
 - `selector`: tagnaam
 - `template/templateUrl`: HTML
 - `styles/styleUrls`: CSS/SCSS

- **view encapsulation:** styles blijven typisch lokaal (niet globaal)
- **Input/Output:**
 - `@Input()` voor data **van parent** → **child**
 - `@Output()` (events) voor **child** → **parent** (emitter)

Wat je praktisch moet kunnen:

- Component opdelen: container (data ophalen) vs presentational (UI).
- Dataflow: parent stuurt props/input, child emit events.
- **Lifecycle hooks (ken de bedoeling):**
 - `ngOnInit`, `ngOnDestroy`, `ngOnChanges`, `ngDoCheck`
 - `ngAfterViewInit/Checked`, `ngAfterContentInit/Checked`

1.4 Templating & databinding

- **Waarom databinding?** Minder foutgevoelig dan manueel DOM lezen/aanpassen met JS.
- **Interpolation:** `{{ variabele }}`
- **Property binding:** `[attr]='value'` (bv. `<img [src]="imageUrl">`)
- **Event binding:** `(event)='handler()'`
- **Two-way binding:** `[(ngModel)]='model'` (vereist `FormsModule`)

Checklist databinding (examengericht):

- Interpolation is **alleen lezen** (van TS → view).
- Property binding is voor properties/attributes (TS → view).
- Event binding is view → TS.
- Two-way is combinatie van beide.

1.5 Directives (view-structuur & styling)

- **Components** zijn ook directives, maar daarnaast heb je:
 - **Structural directives:** veranderen de DOM-structuur
 - `*ngIf`: conditioneel tonen
 - `*ngFor`: itereren (met locals zoals `index`, `first`, `last`, `even`, `odd`)
 - `ngSwitch`, `ngSwitchCase`, `ngSwitchDefault`
 - `ng-template`: template blokken
 - `ng-container`: "wrapper" zonder extra DOM-element
 - **Attribute directives:** wijzigen gedrag/stijl
 - `ngStyle`, `ngClass`

Veelgemaakte fouten:

- `*ngFor` zonder `trackBy` bij grote lijsten kan onnodige re-renders geven.
- `*ngIf` vs "hidden": `*ngIf` maakt DOM echt aan/weg; "hidden" verbergt enkel.

1.6 Async in JS: callbacks → promises → observables

- **Callbacks:** snel "out-of-order code"/callback hell.
- **Promises:** 1 resultaat later (of error).
- **Observables (RxJS):**

- Datastream die **meerdere waarden over tijd** kan emitten.
- In Angular vaak gebruikt voor:
 - HTTP calls naar remote API
 - user events/dialog flows
- Basispatroon:
 - Observable aanmaken (bv. `create`)
 - consumeren met `subscribe`

Wat je minstens moet kennen over RxJS (zonder diepe wiskunde):

- Waarom Observables handig zijn: cancellable, streams, operators.
- Basisoperators die vaak terugkomen: `map`, `filter`, `tap`, `switchMap`, `catchError`, `takeUntil`.
- "Subscribe in component" vs "async pipe" (async pipe is vaak cleaner + auto-unsubscribe).

1.7 Services & Dependency Injection (DI)

- **Service**: logica/data die je over componenten heen wil delen (bv. superhero data).
- Angular werkt met **injectors** en providers.
- **Scopes** die je moet kunnen uitleggen:
 - `root` (app-wide singleton)
 - `module`
 - `component` (nieuwe instantie per component/subtree)
- **providers vs viewProviders**:
 - `providers`: ook beschikbaar voor **projected content** (`<ng-content>`)
 - `viewProviders`: enkel voor de view zelf, niet voor geprojecteerde content

DI-ideetjes die je moet kunnen uitleggen:

- Waarom DI: testbaarheid, losse koppeling, configuratie via providers.
- Scope-effect: root provider = singleton; component provider = nieuwe instantie.

1.8 Angular Material: input componenten

- Material is Google's design language: consistente UI en UX.
- Typische input bouwblokken:
 - `mat-form-field` (appearance, floating labels, hints, placeholders)
 - `MatInputModule`, `MatSelectModule`, chips (multi-select), checkbox (`MatCheckboxModule`)

1.9 Forms in Angular

- **Twee strategieën**:
 - **Template-driven forms**: eenvoudiger, sneller opzetten, "mutable" binding via template.
 - **Reactive forms**: model-driven, beter voor complexe forms, testbaar, logica in TS.
- Template-driven highlights:
 - `FormsModule` nodig
 - `ngForm` + template variabele (bv. `#form="ngForm"`)
 - databinding via `[(ngModel)]`
- Reactive highlight:
 - `valueChanges` is een Observable van user interactions

Wanneer kies je wat?

- Template-driven: kleine forms, simpele validatie, snel UI-first.
- Reactive: complexe forms, dynamische velden, complexe validatie, betere testbaarheid.

Must-know begrippen:

- "Controlled" model: je form state leeft in TS (reactive) vs in template (template-driven).
- Validatie: required/maxLength/pattern + eigen validators.

1.10 Routing & HTTP (typisch in elke Angular app)

- Routing:
 - Declaratief: routes mappen naar componenten.
 - "Route params" (bv. `/items/:id`) en hoe je die uitleest.
 - Guards (hoog niveau): toegang controleren.
- HTTP:
 - `HttpClient` geeft Observables terug.
 - Interceptors (hoog niveau): headers, auth, logging, error handling.
 - CORS: vaak nodig wanneer Angular en API op andere origin draaien.

2) NgRx / Redux voor Angular

2.1 Waarom state management?

- Bij grotere apps wordt "state doorgeven" tussen componenten snel rommelig.
- Redux/NgRx helpt met een **voorspelbare flow**.

2.2 Redux principes

- **Single source of truth**: één Store.
- **State is read-only**: wijzigen kan enkel via **dispatch** van actions.
- **Reducers** (pure functions) berekenen nieuwe state.

2.3 Belangrijke bouwstenen

- **Action**: object met `type` + optioneel `payload`.
- **Reducer**: `(state, action) -> newState` (geen side effects).
 - Klassiek: `switch(action.type)`
 - Modern: `createReducer(...)` met `on(...)`
- **Selector**: Observable die state selecteert/transformeert.
- **Effect**: behandelt side effects/async calls (en mapt acties naar nieuwe acties).

Flow die je moet kunnen tekenen:

- UI → dispatch action
- Reducer → new state
- Selector → UI leest state
- Effect → async → dispatch success/fail action

2.4 Wat hoort in de Store? (SHARI)

Zet vooral state in de Store als die:

- **Shared** is (veel componenten/services)
- **Hydrated** moet worden (persisted in storage)
- **Available** moet blijven bij route re-entry
- **Retrieved** wordt met een side effect
- **Impacted** is door actions van andere bronnen

Niet in de Store:

- niet-gedeelde lokale state
- Angular form state
- niet-serialiseerbare state

Praktische vuistregels:

- Store = "app state", niet "UI micro-state".
- Hou state serialiseerbaar (handig voor debugging, time travel, persistence).
- Reducers mogen geen HTTP calls, timers, random, of mutaties doen.

3) React (basis)

3.1 Waarom React?

- UI bouwen met **herbruikbare componenten**.
- **Virtual DOM**: React vergelijkt oude vs nieuwe virtual DOM (reconciliatie) en update efficiënt.
- Groot ecosysteem (Redux, Next.js, ...) en multi-platform (React Native).

3.2 Setup & build basics

- Voor leren: **create-react-app**.
- Voor productie: frameworks zoals Next.js/Remix/Gatsby/Expo.
- Concepten:
 - **Transpilation** (JSX/nieuw JS → browser JS)
 - **Bundling** (bv. webpack)

3.3 JSX

- JSX lijkt op HTML maar is syntactische sugar die naar JS wordt omgezet.
- Handige patronen:
 - fragments: `<React.Fragment> / <>...</>`
 - JS expressies in `{ ... }`
 - conditions + loops in JSX
 - styles in JSX

3.4 Events

- Click events op knoppen
- Input field content beheren

3.5 Hooks (kern)

- Hook = manier om state/side-effects te gebruiken in function components.
- **Regel:** hooks op top-level callen (niet in loops/conditions).
- Belangrijkste:
 - `useState()`
 - `useEffect()` (setup + cleanup; let op re-renders)
 - `useContext()`, `useRef()`, `useReducer()`

Must-know hooks details:

- `useEffect(fn, deps):`
 - Zonder deps: draait na elke render.
 - Met `[]`: draait 1x na mount (en cleanup bij unmount).
 - Met deps: draait wanneer deps veranderen.
- Cleanup is cruciaal bij timers/subscriptions.

3.6 Mini-case: Task List

- App opdelen in componenten
- Items toevoegen/verwijderen/wijzigen

3.7 JavaScript reminders (moet je echt beheersen)

- `let` vs `var`, `const`
- objects & arrays
- import/export modules
- arrow functions
- promises + `async/await`

4) Angular + ASP.NET (Web API)

4.1 ASP.NET Core Web API + Swagger

- Startpunt: nieuw webapi project (vb. WeatherForecast).
- Swagger/OpenAPI:
 - package: `Swashbuckle.AspNetCore`
 - configuratie in `Program.cs`
 - UI via `.../swagger`

4.2 CRUD met controllers + EF Core (conceptueel)

- Model (vb. `TodoItem`) + DbContext (vb. `TodoContext`).
- DbContext registreren in `Program.cs`.
- Controller genereren via code generator.
- REST best practices:
 - `POST` retournt vaak **Location header** naar `GET /.../{id}`.
 - **Over-posting voorkomen** met **DTO's** (Data Transfer Objects).

HTTP basics die je moet kunnen:

- Status codes: 200/201/204/400/401/403/404/409/500 (wanneer welke?).

- Idempotent: **GET/PUT/DELETE** typisch idempotent, **POST** typisch niet.

4.3 API testen met HttpRepl

- `dotnet tool install -g Microsoft.dotnet-httprepl`
- Connect en test endpoints (bv. `get /api/...`).

4.4 MongoDB variant (BookStore voorbeeld)

- MongoDB = NoSQL document database.
- Basis setup:
 - `mongod --dbpath ...`
 - `mongosh <connectiestring>`
 - `use <db> + db.createCollection(...) + insertMany + find()`
- In ASP.NET:
 - `MongoDB.Driver`
 - `appsettings.json` settings + settings class
 - service laag met CRUD + controller
 - service soms als Singleton geregistreerd

4.5 JSON serialization

- Let op naming (camelCase vs PascalCase).
- Mogelijkheid om property names te mappen met `AddJsonOptions` of `[JsonPropertyName("...")]`.

4.6 Angular app die Web API gebruikt

- Minimale eisen uit de slides:
 - lijst tonen (bv. books)
 - form om item toe te voegen

4.7 SignalR (high level)

- Voor real-time updates (push van server naar clients), typisch voor live data/chat/IoT.

5) Modern Frontend Build Pipeline (leerstofoverzicht)

Een build pipeline zet je project om van "developer-friendly source" naar "browser-friendly output".

- **Input:** TypeScript/ESNext, JSX/TSX, SCSS, assets (images/fonts), configuratie.
- **Output:** geoptimaliseerde JS/CSS bundles (vaak meerdere chunks), assets, en (optioneel) source maps.
- **Twee modi:**
 - **Dev:** snelle rebuilds, duidelijke errors, meestal ruimere source maps.
 - **Prod:** maximale performance: minify, tree-shake, split, compress.

5.1 Kernbegrippen (die je zeker moet kunnen uitleggen)

5.1.1 Transpilation

Transpilieren = syntax omzetten naar een versie die browsers begrijpen.

- Voorbeelden:
 - TypeScript → JavaScript (types verdwijnen volledig)
 - JSX → JavaScript functie-calls
 - Modern JS (ES202x) → oudere JS syntax (indien nodig)
- Tools: Babel, TypeScript compiler, esbuild, swc.

Verschil met "type checking":

- Transpilation kan slagen terwijl type checking faalt (afhankelijk van setup).

5.1.2 Bundling

Bundlen = je imports volgen en samenpakken in 1 of meerdere output-bestanden.

- Waarom:
 - Dependency graph analyseren
 - Optimalisaties mogelijk maken (tree-shaking, splitting)
 - Minder/efficiëntere requests (historisch belangrijk; nog steeds relevant)
- Tools: webpack, Vite (gebruikt Rollup voor build), Rollup, Parcel.

5.1.3 Minification

Minify = code kleiner maken zonder gedrag te veranderen.

- Voorbeelden:
 - whitespace verwijderen
 - variabelen verkorten
 - simpele code transforms
- Tools: Terser, esbuild, swc.

5.1.4 Tree-shaking / Dead Code Elimination

Tree-shaking = ongebruikte exports verwijderen op basis van de import/export-structuur.

- Werkt best met **ES Modules** (`import/export`).
- Kan mislopen bij:
 - modules met **side effects** (code die meteen bij import iets doet)
 - "barrel files" of dynamische import patterns

5.1.5 Code splitting & lazy loading

Code splitting = output opsplitsen in meerdere chunks.

- Doel: kleinere initiële download → sneller eerste scherm.
- Typische triggers:
 - dynamische import (`import(...)`)
 - route-based splitting (Angular routes / React router patterns)

5.1.6 Dev server & HMR

- **Dev server:** serveert lokaal, rebuilds, proxy naar backend.
- **HMR (Hot Module Replacement):** vervangen van modules zonder volledige reload.

5.1.7 Source maps

Source maps mappen de output (minified/transpiled) terug naar je originele source.

- Pro: debugging wordt realistischer.
- Con: kan groot zijn; kan (afhankelijk van config) source prijs geven in productie.

5.2 De pipeline als stappenplan

Je kan een typische pipeline als deze flow uitleggen:

1. **Resolve & parse:** bundler leest entrypoints en volgt imports.
2. **Loaders/plugins:** transpile TS/JSX, compile SCSS → CSS, asset processing.
3. **Optimize:**
 - tree-shake
 - minify
 - split chunks
4. **Emit:** schrijf output naar **dist/**.
5. **Serve** (dev) of **deploy** (prod).

5.3 CSS/SCSS in de pipeline

- SCSS wordt meestal gecompileerd naar CSS (Sass).
- Vaak met:
 - autoprefixer (vendor prefixes)
 - CSS minification
 - (soms) CSS modules of component-scoped styles

Wat je moet kunnen duiden:

- Globale styles vs component styles.
- Waarom preprocessors nuttig zijn (variables, mixins, nesting).

5.4 Assets (images/fonts)

- Pipeline kan assets:
 - kopiëren naar output
 - fingerprints/hashes geven (cache busting)
 - comprimeren
 - inline'en (kleine assets) afhankelijk van config

5.5 Environments & configuratie

- **Environment variables** (dev/prod) sturen:
 - API base URL
 - feature flags
 - logging levels

- Angular heeft typisch verschillende build configs; Vite gebruikt `.env`-varianten.

Belangrijk concept: **build-time** config $\neq$ **runtime** config.

5.6 Performance & caching (must-know)

- **Caching**: browsers cachen assets agressief.
- **Cache busting**: content-hash in filenames (`app.3f2a1c.js`).
- **CDN**: statische assets wereldwijd sneller.

5.7 Quality gates (CI/CD)

Een “professionele” pipeline is niet alleen bundlen:

- Lint (ESLint)
- Format (Prettier)
- Type-check
- Tests (unit/integration)
- Build + artifact
- Deploy

5.8 Typische examen vragen (met korte antwoorden)

- Wat is het verschil tussen transpilation en bundling?
 - Transpilation zet syntax om; bundling volgt imports en maakt output bundles.
- Wat is tree-shaking en wanneer werkt het niet goed?
 - Ongebruikte exports verwijderen; lastig bij side effects en niet-ESM patterns.
- Waarom code splitting?
 - Kleinere initial load, sneller eerste render; laad rest pas als nodig.
- Waarom source maps in dev wel en soms niet in prod?
 - Dev: debugging. Prod: size/security trade-offs.
- Wat doet HMR?
 - Module vervangen zonder full reload, sneller itereren.

5.9 Mini-spiekbrief

- Transpile: TS/JSX $\rightarrow$ JS
- Bundle: imports $\rightarrow$ (meerdere) bundles
- Optimize: tree-shake, split, minify
- Serve dev: dev server + HMR
- Ship prod: hashes + compress + CDN

6) Snelle zelftest (examengericht)

Angular

- Leg het verschil uit tussen interpolation, property binding en two-way binding.
- Wanneer gebruik je `*ngIf` vs `ngSwitch`?
- Waarom zijn Observables handig voor HTTP calls?
- Wat is het verschil tussen `providers` en `viewProviders` (met `<ng-content>`) ?
- Wanneer kies je template-driven forms vs reactive forms?
- Leg het verschil uit tussen "subscribe in component" en "async pipe".

NgRx

- Beschrijf de flow: component → dispatch action → reducer → nieuwe state → selector → UI.
- Waarom moeten reducers "pure" zijn?
- Geef 3 voorbeelden van state die wél in de Store hoort (SHARI) en 3 die niet hoort.
- Welke acties maak je typisch rond async: `load`, `loadSuccess`, `loadFailure`?

React

- Wat is Virtual DOM en wat is reconciliatie?
- Waarom is `useEffect` nodig voor side effects (timer/fetch) en wat doet cleanup?
- Wat is het verschil tussen props en state?
- Geef een voorbeeld van een dependency array bug in `useEffect`.

ASP.NET Web API

- Waarom DTO's gebruiken (over-posting)?
- Wat doet Swagger/OpenAPI voor je API?
- Hoe test je endpoints met HttpRepl?
- In MongoDB: wat is het verschil met relationele DB's (documenten/collections)?

Build pipeline

- Leg uit wat transpilation vs bundling is.
- Wanneer gebruik je code splitting/lazy loading?
- Wat is tree-shaking en wanneer kan het mislopen?